services\orchestrator.py

```
import os
import json
from typing import Dict, List, Any
from datetime import datetime, timezone, timedelta
from collections import Counter
import gc

# Analysis services
from services.analysis.trend_analyzer import get_cve_trends_last_30_days
from services.analysis.severity_analyzer import get_severity_card_counts,
get_severity_percentages
from services.analysis.cwe_processor import get_vendor_risk_analysis,
get_weighted_cwe_analysis

# Data sources
from services.data.api_client import api_client
from services.cache.cache_manager import cache_manager
from services.data.data_source_config import data_source_config
from services.data.data_processor import historical_loader
from database.db_manager import db_manager
import config

class DataOrchestrator:
"""Clean orchestrator using separate visualization modules with database caching and
lazy loading"""
def __init__(self):
data_source_config.log_data_source_status()
self._vendor_risk_cache = None
self._vendor_risk_cache_time = None

# Track loaded data to avoid redundant loads
self._loaded_components = set()

def get_dashboard_data(self, year=None, month=None, day=None, severity_filter=None,
timeline_years=1, load_historical=False):
"""Get all dashboard data with lazy loading and memory optimization"""
print(f"[Orchestrator] Loading dashboard data with filters: year={year}, month={month},
"
f"day={day}, severity={severity_filter}, timeline_years={timeline_years}, "
f"load_historical={load_historical}")

try:
# Always check memory usage before loading data
cache_manager.check_memory_usage()

dashboard_data = {
'metrics': {},
'total_cves': 0,
'severity_percentage': {},
'timeline_data_days': {},
'timeline_data_years': {},
'cwe_radar': {},
'cwe_radar_all': {},
'cwe_radar_weighted': {},
'cwe_radar_descriptions': {},
'latest_cves': [],
'available_years': list(range(datetime.now().year, 2009, -1)),
'available_months': list(range(1, 13)),
'note_text': '',
'historical_loaded': False
}

# 1. CVE Trends (Last 30 days) - always load
print("[Orchestrator] Getting CVE trends (last 30 days) - unfiltered")
dashboard_data['timeline_data_days'] = self._get_always_30_day_trends()

# 2. Vulnerabilities Over Time - only load if requested
if load_historical and timeline_years > 0:
print(f"[Orchestrator] Getting vulnerabilities over time (last {timeline_years} years)")
dashboard_data['timeline_data_years'] = self._get_timeline(timeline_years)
dashboard_data['historical_loaded'] = True
else:
```

```
print("[Orchestrator] Skipping historical timeline (not requested)")
dashboard_data['timeline_data_years'] = {'labels': [], 'values': [],
'total_cves': 0, 'months_covered': 0,
'raw_data': {}}

# 3. Filtered data for Cards and Pie Chart ONLY
print("[Orchestrator] Getting filtered data for cards and pie chart")
filtered_cves = self.get_filtered_cves(year=year, month=month, day=day)

# Apply severity filter if provided
if severity_filter:
from utils.filters import FilterService
filter_service = FilterService()
filtered_cves = filter_service.filter_by_severity(filtered_cves, severity_filter)

# 4. Calculate severity counts
severity_counts = Counter()
for cve in filtered_cves:
severity = cve.get('Severity', 'UNKNOWN').upper()
if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
severity_counts[severity] += 1
else:
severity_counts['UNKNOWN'] += 1

severity_metrics = {
'CRITICAL': severity_counts.get('CRITICAL', 0),
'HIGH': severity_counts.get('HIGH', 0),
'MEDIUM': severity_counts.get('MEDIUM', 0),
'LOW': severity_counts.get('LOW', 0),
'UNKNOWN': severity_counts.get('UNKNOWN', 0),
'total_cves': len(filtered_cves)
}

total = len(filtered_cves)
if total > 0:
severity_percentages = {}
for key in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']:
percentage = round((severity_counts.get(key, 0) * 100 / total))
severity_percentages[key] = f"{percentage}%"
else:
severity_percentages = {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW',
'UNKNOWN']}

# 5. Vendor Risk Analysis - Cache for 24 hours
# Only load if not in learn pages (to save memory)
# Try to load from database cache first
vendor_risk_key = 'vendor_risk_analysis'
weighted_vendor_risk_key = 'weighted_vendor_risk_analysis'

# Try to get from database
vendor_risk = None
weighted_vendor_risk = None

if db_manager.use_database:
vendor_risk_data = db_manager.get_summary_stats(vendor_risk_key)
weighted_vendor_risk_data = db_manager.get_summary_stats(weighted_vendor_risk_key)

if vendor_risk_data and 'data' in vendor_risk_data:
vendor_risk = vendor_risk_data['data']
print(f"[Orchestrator] Using vendor risk analysis from database cache")

if weighted_vendor_risk_data and 'data' in weighted_vendor_risk_data:
weighted_vendor_risk = weighted_vendor_risk_data['data']
print(f"[Orchestrator] Using weighted vendor risk analysis from database cache")

# Fall back to memory cache if needed
if not vendor_risk or not weighted_vendor_risk:
cache_expired = False
if self._vendor_risk_cache_time:
cache_age = datetime.now() - self._vendor_risk_cache_time
if cache_age > timedelta(hours=24):
cache_expired = True
print("[Orchestrator] Vendor risk cache expired (>24 hours old)")

if self._vendor_risk_cache and not cache_expired:
cache_age_str = ""
if self._vendor_risk_cache_time:
```

```
age = datetime.now() - self._vendor_risk_cache_time
hours = int(age.total_seconds() / 3600)
cache_age_str = f" (cached {hours}h ago)"
print(f"[Orchestrator] Using cached vendor risk analysis{cache_age_str}")
vendor_risk, weighted_vendor_risk = self._vendor_risk_cache
else:
print("[Orchestrator] Loading vendor risk analysis for 2025 (will cache for 24 hours)")
try:
# Only analyze current year to save memory
vendor_risk = get_vendor_risk_analysis()
weighted_vendor_risk = get_weighted_cwe_analysis()

# Save to memory cache
self._vendor_risk_cache = (vendor_risk, weighted_vendor_risk)
self._vendor_risk_cache_time = datetime.now()

# Save to database cache if available
if db_manager.use_database:
db_manager.save_summary_stats(vendor_risk_key, 1, vendor_risk)
db_manager.save_summary_stats(weighted_vendor_risk_key, 1, weighted_vendor_risk)

print("[Orchestrator] Vendor risk cached (expires in 24 hours)")
except Exception as e:
print(f"[Orchestrator] Error loading vendor risk analysis: {e}")
vendor_risk = self._get_empty_vendor_risk()
weighted_vendor_risk = {'indices': [], 'labels': [], 'values': []}

# Build note text for UI
cutoff_info, explanation = data_source_config.get_data_source_cutoff()
if year and month and day:
note_text = f"Cards & Pie Chart: {year}-{month:02d}-{day:02d} | Trends: Last 30 days"
elif year and month:
note_text = f"Cards & Pie Chart: {year}-{month:02d} | Trends: Last 30 days"
elif year:
note_text = f"Cards & Pie Chart: {year} data | Trends: Last 30 days"
elif severity_filter:
note_text = f"Cards & Pie Chart: {severity_filter.lower()} severity | Trends: Last 30
days"
else:
note_text = f"Cards & Pie Chart: Last 30 days | Trends: Last 30 days"

if load_historical:
note_text += f" | Timeline: Last {timeline_years} year(s)"

# Update dashboard data
dashboard_data.update({
'metrics': severity_metrics,
'total_cves': len(filtered_cves),
'severity_percentage': severity_percentages,
'cwe_radar': vendor_risk.get('top_10_cwes', {'indices': [], 'labels': [], 'values':
[]}),
'cwe_radar_all': vendor_risk,
'cwe_radar_weighted': weighted_vendor_risk,
'cwe_radar_descriptions': self._get_cwe_descriptions(),
'latest_cves': filtered_cves[:100],  # Only return first 100 to save memory
'note_text': note_text,
'historical_loaded': load_historical
})

# Free up memory after building the data
filtered_cves = None
gc.collect()

print(f"[Orchestrator] Data loaded successfully:")
print(f"  - Filtered CVEs (cards/pie): {dashboard_data['total_cves']}")
print(f"  - CVE Trends data points:
{len(dashboard_data['timeline_data_days'].get('labels', []))}")
print(f"  - Timeline data points:
{len(dashboard_data['timeline_data_years'].get('labels', []))}")
print(f"  - Severity counts: C={severity_metrics['CRITICAL']}, "
f"H={severity_metrics['HIGH']}, M={severity_metrics['MEDIUM']}, "
f"L={severity_metrics['LOW']}")

print(f"[Orchestrator] Dashboard data structure created successfully")
return dashboard_data

except Exception as e:
```

```
print(f"[Orchestrator] Error in get_dashboard_data: {e}")
import traceback
traceback.print_exc()
raise e

def _get_empty_vendor_risk(self):
return {
'top_5_cwes': {'indices': [], 'labels': [], 'values': []},
'top_10_cwes': {'indices': [], 'labels': [], 'values': []},
'all_cwes': {'indices': [], 'labels': [], 'values': []},
'severity_matrix': {},
'cwe_30_day_counts': {},
'total_cves_analyzed': 0,
'years_analyzed': []
}

def _get_always_30_day_trends(self) -> Dict[str, Any]:
"""Get CVE trends for the last 30 days - Database optimized"""
try:
# Try to get from database cache
if db_manager.use_database:
trends_data = db_manager.get_summary_stats('cve_trends_30_days')
if trends_data and 'data' in trends_data:
last_updated = trends_data.get('last_updated')
if last_updated:
# Check if cache is still valid (less than 24 hours old)
if isinstance(last_updated, datetime):
age = datetime.now() - last_updated
else:
# Convert string to datetime
last_updated_dt = datetime.fromisoformat(str(last_updated).replace('Z', '+00:00'))
age = datetime.now() - last_updated_dt

if age.total_seconds() < 24 * 60 * 60:
print(f"[Orchestrator] Using cached 30-day trends from database")
return trends_data['data']

# Fall back to generating new data
trends_data = get_cve_trends_last_30_days()

# Cache in database
if db_manager.use_database:
db_manager.save_summary_stats('cve_trends_30_days',
len(trends_data.get('labels', [])),
trends_data)

return trends_data
except Exception as e:
print(f"[Orchestrator] Error getting 30-day trends: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'date_range': {'start': '', 'end':
''}}

def _get_timeline(self, years=1) -> Dict[str, Any]:
"""Get vulnerability timeline data - Database optimized"""
try:
# Try to get from database first
if db_manager.use_database:
timeline_data = historical_loader.calculate_vulnerabilities_timeline(years)
if timeline_data and timeline_data.get('labels'):
return timeline_data

# Fall back to the analyzer if needed
from services.analysis.trend_analyzer import get_vulnerabilities_over_time_last_n_years
return get_vulnerabilities_over_time_last_n_years(years)
except Exception as e:
print(f"[Orchestrator] Error getting timeline: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0, 'raw_data':
{}}

def get_filtered_cves(self, year=None, month=None, day=None) -> List[Dict[str, Any]]:
"""Get CVEs with filtering - Database optimized"""
print(f"[Orchestrator] Getting filtered CVEs: year={year}, month={month}, day={day}")

cutoff_info, explanation = data_source_config.get_data_source_cutoff()

# If no year specified, get recent CVEs from API
if not year:
```

```
print(f"[Orchestrator] Using API data for recent data")
cves = api_client.get_cves_last_30_days()
print(f"[Orchestrator] API returned {len(cves)} CVEs")
return cves

# Check if we should use historical data or API data
if data_source_config.should_use_historical(year, month):
print(f"[Orchestrator] Using HISTORICAL data for {year}-{month or 'all'}")

# Try database first
if db_manager.use_database:
cves = db_manager.get_cves_by_filter(year=str(year),
month=str(month) if month else None,
day=str(day) if day else None)
if cves:
print(f"[Orchestrator] Retrieved {len(cves)} CVEs from database")
return cves

# Fall back to historical data processor
return historical_loader.get_filtered_cves_for_year(
year=year,
month=month,
day=day
)
else:
print(f"[Orchestrator] Using API data for {year}-{month or 'all'}")

# Get all CVEs from last 30 days and filter them manually
cves = api_client.get_cves_last_30_days()

# Filter the CVEs by date
filtered_cves = []
for cve in cves:
pub_date = cve.get('Published', '')
if pub_date:
try:
# Parse date from various formats
if 'T' in pub_date:
dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
else:
dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

# Match year
if int(year) == dt.year:
# Match month if provided
if month is None or int(month) == dt.month:
# Match day if provided
if day is None or int(day) == dt.day:
filtered_cves.append(cve)
except:
continue

print(f"[Orchestrator] Filtered API data: {len(filtered_cves)} CVEs match {year}-{month
or 'all'}-{day or 'all'}")
return filtered_cves

def get_cve_detail(self, cve_id: str) -> Dict[str, Any]:
"""Get details for a specific CVE"""
# Try database first
if db_manager.use_database:
cves = db_manager.get_cves_by_filter(limit=1)
for cve in cves:
if cve.get('ID') == cve_id:
print(f"[Orchestrator] Found CVE {cve_id} in database")
return cve

# Fall back to API
return api_client.get_cve_detail(cve_id)

def clear_cache(self):
"""Clear all caches"""
api_client.clear_cache()
cache_manager.clear_cache()
historical_loader.clear_cache()
self._vendor_risk_cache = None
self._vendor_risk_cache_time = None
print("[Orchestrator] All caches cleared")
```

```
def get_data_source_status(self) -> Dict[str, Any]:
"""Get status of data sources"""
status = data_source_config.get_status()

# Add database status
if db_manager.use_database:
db_stats = db_manager.get_stats()
status.update({
'database_enabled': True,
'database_stats': db_stats
})
else:
status.update({
'database_enabled': False,
'database_stats': {}
})

# Add memory cache status
cache_stats = cache_manager.get_cache_stats()
status['cache_stats'] = cache_stats

return status

def _get_cwe_descriptions(self) -> Dict[str, str]:
"""Get CWE descriptions - static version to save memory"""
return {
"CWE-79": "Improper Neutralization of Input During Web Page Generation ('Cross-site
Scripting')",
"CWE-89": "Improper Neutralization of Special Elements used in an SQL Command ('SQL
Injection')",
"CWE-20": "Improper Input Validation",
"CWE-22": "Improper Limitation of a Pathname to a Restricted Directory ('Path
Traversal')",
"CWE-119": "Improper Restriction of Operations within the Bounds of a Memory Buffer",
"CWE-200": "Exposure of Sensitive Information to an Unauthorized Actor",
"CWE-287": "Improper Authentication",
"CWE-352": "Cross-Site Request Forgery (CSRF)",
"CWE-74": "Improper Neutralization of Special Elements in Output Used by a Downstream
Component ('Injection')",
"CWE-862": "Missing Authorization",
"CWE-125": "Out-of-bounds Read",
"CWE-284": "Improper Access Control",
"CWE-416": "Use After Free",
"CWE-787": "Out-of-bounds Write"
}

# Global orchestrator instance
data_orchestrator = DataOrchestrator()
```